

Module 7 — Spring Boot

Highest priority. The “how does auto-configuration work” and “trace a request through the DispatcherServlet” questions are asked at every level. Know the request flow cold.

Node.js bridge: Spring Boot ≈ an opinionated, batteries-included Express/Nest setup: embedded server (like `app.listen()`), auto-wired middleware, config profiles, health endpoints — minus the manual plumbing.

7.1 Spring Boot Architecture & Auto-Configuration

1. Why Interviewers Ask This

Auto-configuration is *the* Spring Boot feature. “What is `@SpringBootApplication`?” and “how does auto-config decide what to configure?” are guaranteed.

2. Core Concept

Spring Boot = Spring + **auto-configuration** + **starters** + **embedded server** + **opinionated defaults**. It removes boilerplate XML/config so you “just run”.

`@SpringBootApplication` = `@Configuration` + `@ComponentScan` + `@EnableAutoConfiguration`.

3. Internal Working — auto-configuration mechanism

1. `@EnableAutoConfiguration` triggers `AutoConfigurationImportSelector`
2. It reads `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (older: `spring.factories`) from every starter jar → list of `@AutoConfiguration` classes
3. Each auto-config class is GUARDED by `@Conditional` annotations:
 - `@ConditionalOnClass` (is H2/Tomcat/Jackson on the classpath?)
 - `@ConditionalOnMissingBean` (did the user NOT define their own?)
 - `@ConditionalOnProperty` (is a property set?)
4. Conditions that pass → beans registered (`DataSource`, `DispatcherServlet`, `ObjectMapper`...)
5. Your beans always win (`@ConditionalOnMissingBean` backs off if you defined one)

So “convention over configuration” = *classpath presence* + *conditions* decide what gets wired.

4. Memory Diagram

```
classpath has spring-boot-starter-web
-> Tomcat + Spring MVC jars present
-> WebMvcAutoConfiguration @ConditionalOnClass(DispatcherServlet) PASSES
-> registers DispatcherServlet, Jackson ObjectMapper, embedded Tomcat
(unless you defined your own -> @ConditionalOnMissingBean backs off)
```

5. Starter Dependencies

Curated, version-aligned dependency bundles: `spring-boot-starter-web` (MVC + Tomcat + Jackson), `-data-jpa` (Hibernate + JPA), `-security`, `-actuator`, `-test`. The **parent BOM** manages compatible versions so you don't resolve conflicts manually.

6. Most Asked Questions

- What is `@SpringBootApplication`? (3 annotations)
- How does auto-configuration work? (*imports* + `@Conditional`)
- What are starters? Why? (*curated, versioned dependency sets*)
- How do you override an auto-configured bean? (*define your own* → `@ConditionalOnMissingBean` backs off)
- How to exclude an auto-config? (`@SpringBootApplication(exclude=DataSourceAutoConfiguration.class)`)
- How to debug what got auto-configured? (`--debug` → *Conditions Evaluation Report*)

7. Traps

- Saying auto-config is “magic” — it’s conditional bean registration from imports files.
- Thinking starters contain code (they’re mostly dependency aggregators).
- Not knowing your own bean overrides the auto-configured one.

8. Best Answer

“@SpringBootApplication bundles @Configuration, @ComponentScan, and @EnableAutoConfiguration. At startup, AutoConfigurationImportSelector loads auto-config classes listed in each starter’s imports file. Each is guarded by @Conditional checks — @ConditionalOnClass, @ConditionalOnMissingBean, @ConditionalOnProperty — so beans are only created when the right libraries are present and I haven’t defined my own. That’s why adding starter-web gives me an embedded Tomcat and JSON mapping with zero config, but my own ObjectMapper bean always wins.”

9. Coding Example

```
@SpringBootApplication
public class App {
    public static void main(String[] args) { SpringApplication.run(App.class, args); }

    @Bean
    // overrides Boot's auto-configured ObjectMapper
    ObjectMapper objectMapper() {
        return new ObjectMapper().findAndRegisterModules()
            .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
    }
}
```

10. Follow-ups

- Write a custom auto-configuration with @ConditionalOnProperty.
- How does the embedded Tomcat start? (ServletWebServerFactory bean)

11 & 12. Summary + Cheat

Auto-config = imports files + @Conditional. Starters = versioned deps. Your bean wins.

7.2 DispatcherServlet & the Request Lifecycle (know this cold)

Core Concept

The **DispatcherServlet** is the **front controller** — a single servlet that receives every HTTP request and orchestrates handling.

Internal Working — full request flow

```
HTTP request
-> Servlet container (embedded Tomcat) -> Filter chain (Security, encoding, CORS)
-> DispatcherServlet
    1. HandlerMapping      : find controller method for URL+verb (@RequestMapping)
    2. HandlerAdapter      : invoke it
    3. HandlerInterceptor  : preHandle()
    4. Argument resolvers  : bind @RequestBody/@PathVariable/@RequestParam (+ validation)
    5. Controller method runs -> returns object / ResponseEntity
    6. HttpMessageConverter : serialize return value to JSON (Jackson) [@ResponseBody path]
      (or ViewResolver -> render a view, for @Controller)
    7. HandlerInterceptor  : postHandle / afterCompletion
    8. @ExceptionHandler / HandlerExceptionResolver if anything threw
-> Filter chain (response) -> Tomcat -> HTTP response
```

Memory Diagram

```
[Client] -> [Tomcat] -> [Filters] -> [DispatcherServlet]
                                   |-> HandlerMapping -> Controller
                                   |-> MessageConverter (JSON)
                                   |-> ExceptionResolver
                                   <- response <-----+>
```

Best Answer

“Every request hits the embedded Tomcat, passes the servlet filter chain (security, CORS), then the `DispatcherServlet` front controller. It uses a `HandlerMapping` to find the `@RequestMapping` method, a `HandlerAdapter` to invoke it, argument resolvers to bind and validate the body/params, runs my controller, and an `HttpMessageConverter` (Jackson) to serialize the return value to JSON. Exceptions are routed to `@ExceptionHandler`. Filters are outside the `DispatcherServlet`; interceptors are inside it.”

Filter vs Interceptor (common follow-up): Filters are servlet-level (before/after `DispatcherServlet`, work on raw request/response, e.g., security, logging). Interceptors are Spring MVC-level (have access to the handler/model, run around controller execution).

7.3 Configuration — properties, YAML, Profiles, @ConfigurationProperties

- **application.properties / application.yml** — externalized config. YAML is hierarchical/nicer for nested config; both are equivalent.
- **Profiles** — environment-specific config: `application-dev.yml`, `application-prod.yml`; activate with `spring.profiles.active=prod` (env var / CLI). `@Profile("prod")` conditionally registers beans.
- **Property precedence** (high → low): command-line args → OS env vars → profile-specific files → `application.yml` → defaults. (Know that env/CLI override files.)
- **@Value("\${db.url}")** — inject a single property.
- **@ConfigurationProperties(prefix="app")** — type-safe binding of a group of properties to a POJO (preferred for structured config; supports validation and relaxed binding).

```
@ConfigurationProperties(prefix = "payment")
@Validated
public record PaymentProps(@NotBlank String apiKey, @Min(1) int timeoutMs) {}
// payment.api-key / payment.timeout-ms bind automatically (relaxed binding)
```

@Value vs @ConfigurationProperties: single value + SpEL vs grouped, type-safe, validated, relaxed-binding. Prefer `@ConfigurationProperties` for anything structured.

7.4 REST APIs, Validation, Exception Handling, Logging, Actuator

REST controllers

```
@RestController
@RequestMapping("/api/v1/users")
class UserController {
    private final UserService service;
    UserController(UserService service){ this.service = service; }

    @GetMapping("/{id}")
    User get(@PathVariable Long id){ return service.get(id); }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    User create(@Valid @RequestBody CreateUserRequest req){ return service.create(req); }

    @GetMapping
    Page<User> list(@RequestParam(defaultValue="0") int page,
                  @RequestParam(defaultValue="20") int size){
        return service.list(PageRequest.of(page, size));
    }
}
```

Validation

`@Valid` + Bean Validation (`jakarta.validation`) annotations (`@NotNull`, `@NotBlank`, `@Email`, `@Min`, `@Size`).
Invalid `@RequestBody` → `MethodArgumentNotValidException` → map to **400**.

Exception handling (production pattern)

`@RestControllerAdvice` + `@ExceptionHandler` for centralized error responses (see Module 4).

Logging

SLF4J API + Logback (default). Configure levels per package in properties. Use structured/JSON logs + correlation ids (MDC) in microservices.

Actuator (operational endpoints)

`/actuator/health` (liveness/readiness for k8s), `/metrics`, `/info`, `/env`, `/loggers` (change log level at runtime), `/prometheus` (with Micrometer). Expose selectively and secure them — never expose `/env`/`/heapdump` publicly.

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      probes:
        enabled: true    # /health/liveness & /health/readiness
```

Module 7 — Top 25 Interview Questions (senior answers)

1. **What is Spring Boot vs Spring?** Opinionated auto-config + starters + embedded server on top of Spring.
2. **@SpringBootApplication** =? @Configuration + @ComponentScan + @EnableAutoConfiguration.
3. **How does auto-configuration work?** Imports files + @Conditional guards.
4. **@ConditionalOnClass/OnMissingBean/OnProperty**? Presence/user-override/property gating.
5. **How to override an auto-configured bean?** Define your own → OnMissingBean backs off.
6. **Exclude auto-config?** `exclude = DataSourceAutoConfiguration.class`.

7. **What are starters?** Curated versioned dependency bundles.
8. **Embedded server?** Tomcat by default (Jetty/Undertow optional); no WAR needed.
9. **What is the DispatcherServlet?** Front controller orchestrating request handling.
10. **Full request lifecycle?**
Tomcat → filters → DispatcherServlet → HandlerMapping → adapter → controller → MessageConverter → response.
11. **Filter vs Interceptor?** Servlet-level vs Spring MVC-level (handler-aware).
12. **How is JSON produced?** HttpMessageConverter (Jackson) on @ResponseBody.
13. **@Controller vs @RestController?** View vs body (JSON).
14. **application.properties vs yml?** Flat vs hierarchical; equivalent.
15. **Profiles?** Env-specific config/beans via `spring.profiles.active` + `@Profile`.
16. **Property precedence?** CLI > env > profile file > application.yml > defaults.
17. **@Value vs @ConfigurationProperties?** Single/SpEL vs grouped/type-safe/validated.
18. **How do you validate requests?** `@Valid` + Bean Validation → 400 on failure.
19. **Centralized exception handling?** `@RestControllerAdvice` + `@ExceptionHandler`.
20. **What is Actuator?** Production endpoints: health, metrics, info, loggers.
21. **k8s probes?** `/health/liveness` & `/health/readiness`.
22. **How to see what auto-configured?** Run with `--debug` → conditions report.
23. **How does Boot start Tomcat?** `ServletWebServerFactory` bean during context refresh.
24. **CommandLineRunner/ApplicationRunner?** Run code after startup.
25. **How to build a runnable artifact?** Fat/uber JAR via `spring-boot-maven-plugin`.

Module 7 — Top Coding Questions

- Build a paginated, validated CRUD REST controller + global exception handler.
- Bind config with `@ConfigurationProperties` + validation.
- Add a custom health indicator and a Micrometer metric.
- Write a `HandlerInterceptor` (or Filter) that logs latency + correlation id.

Module 7 — Common Follow-ups

- “Where exactly is the request body deserialized and validated?”
- “How would you run different config in dev vs prod?”
- “How do you expose metrics to Prometheus safely?”

Module 7 — One-Page Cheat Sheet

```

Boot = Spring + auto-config + starters + embedded Tomcat + defaults
@SpringBootApplication = @Configuration + @ComponentScan + @EnableAutoConfiguration
Auto-config: AutoConfiguration.imports + @ConditionalOnClass/OnMissingBean/OnProperty; your bean wins
Request: Tomcat -> Filters -> DispatcherServlet -> HandlerMapping -> HandlerAdapter ->
        controller -> HttpMessageConverter(Jackson) -> response ; @ExceptionHandler on error
Filter=servlet level ; Interceptor=MVC handler level
Config: yml/properties, profiles (spring.profiles.active), precedence CLI>env>profile>file
@Value(single) vs @ConfigurationProperties(grouped/typed/validated)
Validation: @Valid -> MethodArgumentNotValidException -> 400
Actuator: /health(liveness,readiness) /metrics /info /loggers /prometheus (secure them!)

```

Module 7 — Mock Interview (answer, then continue)

1. "Trace a `POST /api/v1/users` with a JSON body from socket to response, naming every component."
2. "Explain auto-configuration deeply — how does Boot decide to create a `DataSource`?"
3. "How do you override the default Jackson `ObjectMapper`, and why does yours win?"
4. "Dev uses H2, prod uses Postgres — how do you configure that cleanly?"
5. "Which Actuator endpoints do you expose in prod and how do you secure them?"

Continue to Module 8 when ready.